Patrones creacionales

Capturas de la clase

Unidad 3 - Patrones de diseño | 04/11/2026

Documento generado a partir de 17 capturas de pantalla tomadas en clase.
El orden de las capturas corresponde al orden en que fueron tomadas.
Cada captura ocupa una pagina; el indice siguiente indica el archivo de origen.

Fuente

`Patrones creacionales.png ... Patrones creacionales-16.png`

Documento de apoyo académico. El contenido de las capturas es material de la cátedra; este PDF solo las ordena y las pagina para su consulta.

Índice de capturas

#	Archivo de origen	Página
1	Patrones creacionales.png	3
2	Patrones creacionales-1.png	4
3	Patrones creacionales-2.png	5
4	Patrones creacionales-3.png	6
5	Patrones creacionales-4.png	7
6	Patrones creacionales-5.png	8
7	Patrones creacionales-6.png	9
8	Patrones creacionales-7.png	10
9	Patrones creacionales-8.png	11
10	Patrones creacionales-9.png	12
11	Patrones creacionales-10.png	13
12	Patrones creacionales-11.png	14
13	Patrones creacionales-12.png	15
14	Patrones creacionales-13.png	16
15	Patrones creacionales-14.png	17
16	Patrones creacionales-15.png	18
17	Patrones creacionales-16.png	19


CLASE INVERTIDA
Patrones creacionales
1 Builder
2 Singleton
3 Factory Method
4 Abstract Factory
David Jurado · Diseño de Software · ESPOL
1

RETO BIBLIOTECA UNIVERSITARIA

La app web y móvil crece

Este bloque se repite en **14 clases**:

```
if (plataforma.equals("web")) {
    boton   = new BotonBuscarWeb();
    tarjeta = new TarjetaLibroWeb();
} else if (plataforma.equals("movil")) {
    boton   = new BotonBuscarMovil();
    tarjeta = new TarjetaLibroMovil();
}
```


RETO
BIBLIOTECA UNIVERSITARIA
La app web y móvil crece
Este bloque se repite en 14 clases:
if (plataforma.equals("web")) {
boton = new BotonBuscarWeb();
tarjeta = new TarjetaLibroWeb();
} else if (plataforma.equals("movil")) {
boton = new BotonBuscarMovil();
tarjeta = new TarjetaLibroMovil();
}
Llega una tercera plataforma: el kiosko de autoservicio.
1 ¿Cuántos archivos tocas?
2 ¿Qué principio SOLID se rompe?
David Jurado · Diseño de Software · ESPOL
3

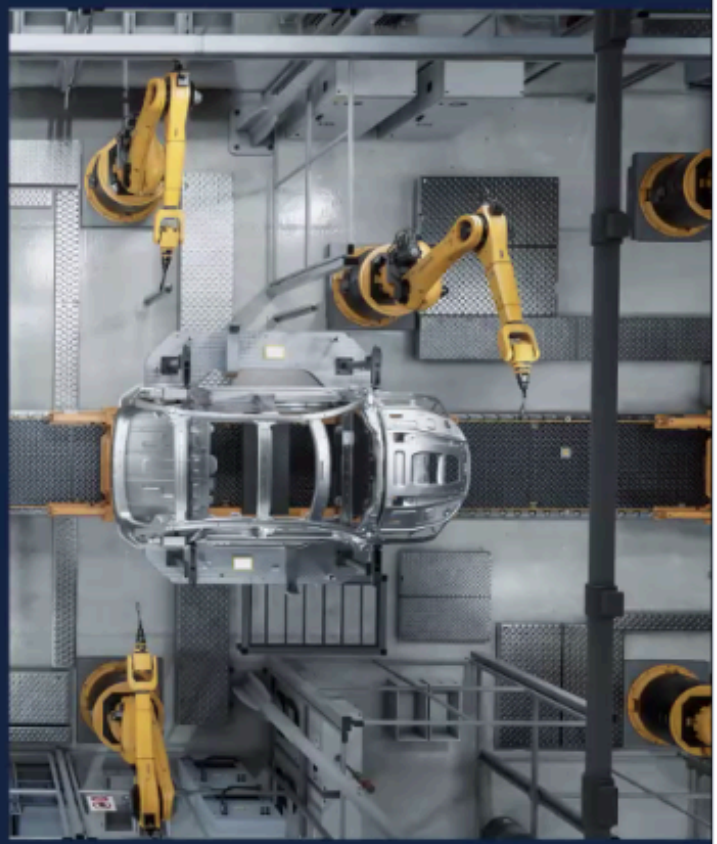

PANORAMA
¿Qué tienen en común los patrones creacionales?
1
Encapsulan el new
El cliente pide un objeto; no sabe qué clase lo crea.
2
Clase vs. objeto
De clase: por herencia. De objeto: por delegación.
3
Puente con SOLID
OCP: extender sin editar. DIP: depender de abstracciones.
David Jurado · Diseño de Software · ESPOL
4

1

PATRÓN 1 DE 4

Builder

Intención: separar la construcción de un objeto complejo de su representación.



David Jurado · Diseño de Software · ESPOL

5

SITUACIÓN BUILDER

Un pedido en la cafetería del campus

El sistema registra una bebida personalizada así:

```
new Bebida("latte", "grande", null, true, false, 2, null, "canela");
```

? ¿Qué significa cada argumento?

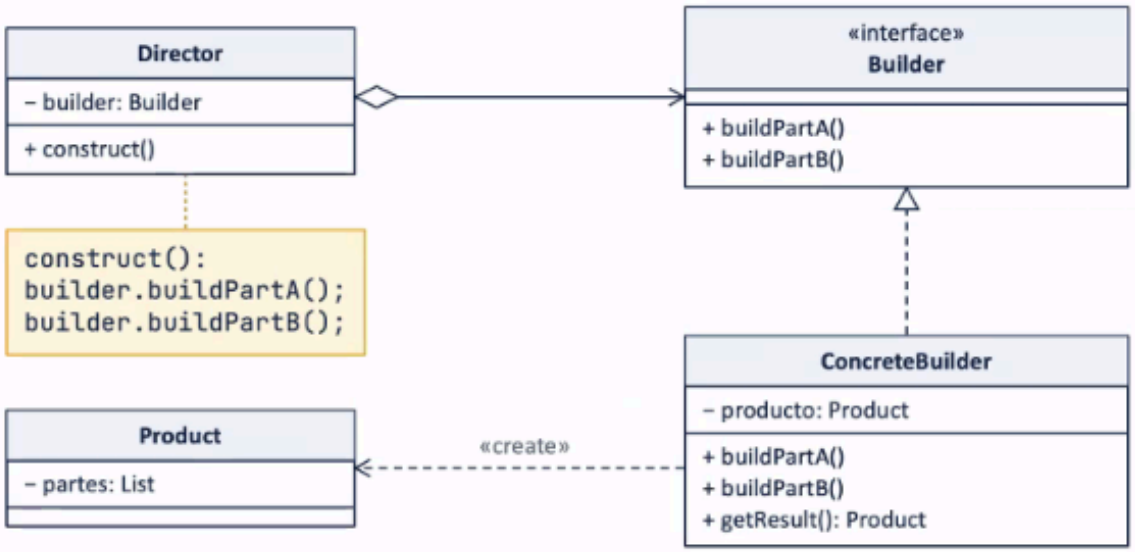
TEORÍA PARA RESALTAR BUILDER

Builder en cuatro ideas

1. El **Director** decide el orden.
2. El **Builder** arma una parte por llamada y guarda el estado intermedio.
3. El producto se entrega al final: **getResult()**.
4. Mismo proceso, **distintas representaciones**.

DIAGRAMA UML BUILDER

Builder: diagrama de clases



EN LA CAFETERÍA

Director
Receta del barista

ConcreteBuilder
BebidaBuilder

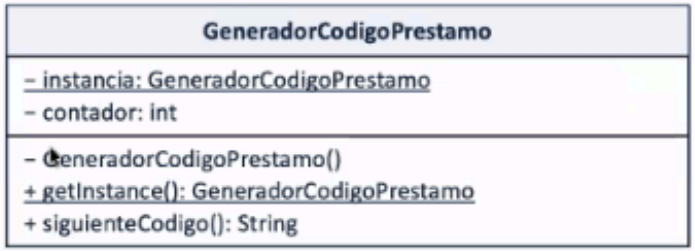
Product
Bebida

Notación UML Herencia · Implementación · «create» · Agregación · + público · – privado · static · *abstracto*

David Jurado · Diseño de Software · ESPOL 8

DIAGRAMA UML SINGLETON

Singleton: diagrama y código



Un solo generador para toda la biblioteca:

```
public class GeneradorCodigoPrestamo {
    private static GeneradorCodigoPrestamo instancia;
    private GeneradorCodigoPrestamo() { }
    public static synchronized
        GeneradorCodigoPrestamo getInstance() {
        if (instancia == null)
            instancia = new GeneradorCodigoPrestamo();
        return instancia;
    }
}
```

Notación UML —▷ Herencia ---▷ Implementación ----> «create» ◇—▷ Agregación + público · – privado static · *abstracto*

David Jurado · Diseño de Software · ESPOL 13

ERROR FRECUENTE SINGLETON

«Singleton para todo»

```
Config.getInstance()
Catalogo.getInstance()
Usuario.getInstance()
```

Es una variable global con otro nombre.

Alternativa: el servicio recibe el generador por parámetro.

```
class ServicioPrestamo {
    private final GeneradorCodigoPrestamo gen;
    ServicioPrestamo(GeneradorCodigoPrestamo g) {
        gen = g;
    }
}
```


3
PATRÓN 3 DE 4
«CONSTRUCTOR VIRTUAL»
Factory Method
Intención: definir una interfaz para crear un objeto y dejar que las subclases decidan qué clase instanciar.
David Jurado · Diseño de Software · ESPOL
15

SITUACIÓN FACTORY METHOD

Reportes de préstamos

Hoy la biblioteca solo exporta PDF:

```
class Exportador {
    void exportar(List<Prestamo> datos) {
        formatear(datos);
        Archivo a = new ArchivoPDF();
        a.escribir(datos);
        guardar(a);
    }
}
```

Los pasos nunca cambian: formatear, generar, guardar.

? **¿Cómo agregas CSV y Excel sin editar la clase?**

David Jurado · Diseño de Software · ESPOL 16

TEORÍA PARA RESALTAR FACTORY METHOD

Factory Method en cuatro ideas

1. Las **subclases** deciden qué crear.
2. Creación por **herencia**.
3. Primo de **Template Method**.
4. Más que un constructor: tiene **nombre** y devuelve **subtipos**.

EN LA LECTURA

Prensa de moldeo

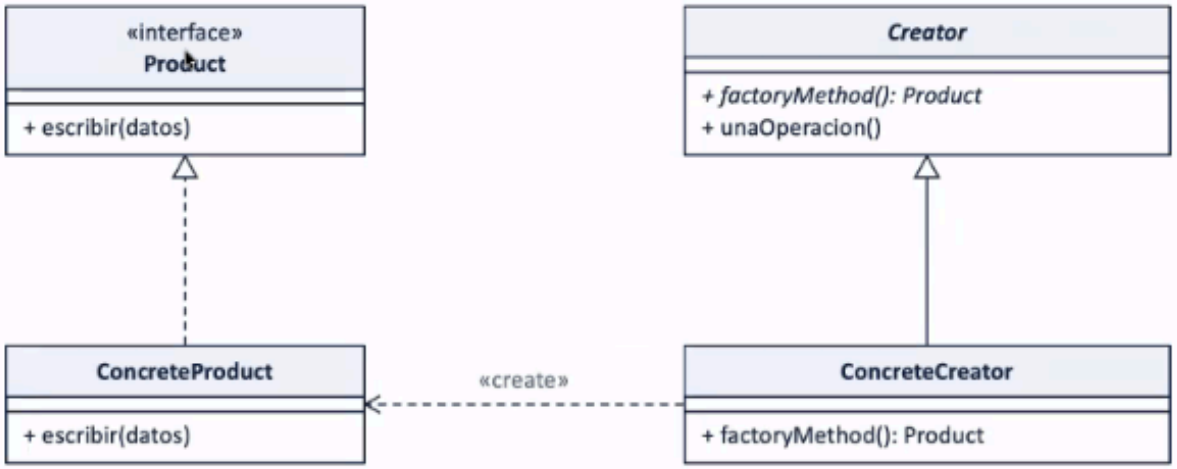
La misma prensa; el molde decide el juguete.

David Jurado · Diseño de Software · ESPOL

17

DIAGRAMA UML FACTORY METHOD

Factory Method: diagrama de clases



EN LA BIBLIOTECA

Creator
Exportador

ConcreteCreator
ExportadorCSV

Product
Archivo

unaOperacion()
exportar()

Notación UML Herencia · Implementación · «create» · Agregación · + público · – privado · static · *abstracto*

David Jurado · Diseño de Software · ESPOL 18


4
PATRÓN 4 DE 4
Abstract Factory
Intención: crear familias de objetos relacionados sin especificar sus clases concretas.
David Jurado · Diseño de Software · ESPOL
20


VUELTA AL RETO
ABSTRACT FACTORY
La biblioteca como matriz
Plataforma
BotonBuscar
TarjetaLibro
FormularioPrestamo
Web
BotonBuscarWeb
TarjetaLibroWeb
FormularioPrestamoWeb
Móvil
BotonBuscarMovil
TarjetaLibroMovil
FormularioPrestamoMovil
Kiosko
BotonBuscarKiosko
TarjetaLibroKiosko
FormularioPrestamoKiosko
?
Agregar una fila es fácil. ¿Y una columna?
David Jurado · Diseño de Software · ESPOL
21

DIAGRAMA UML ABSTRACT FACTORY

Abstract Factory: diagrama de clases

